

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ  
КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ імені Тараса Шевченка  
ФАКУЛЬТЕТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ  
**Кафедра програмних систем і технологій**

Дисципліна  
**«Конструювання програмного забезпечення»**

**Лабораторна робота № 3**  
**Варіант 1**

|                |                                 |                |                              |
|----------------|---------------------------------|----------------|------------------------------|
| Виконав:       | Приймаченко<br>Артем Андрійович | Перевірів:     | Шабатін Павло<br>Євгенійович |
| Група          | ІПЗ-34-10                       | Дата перевірки |                              |
| Форма навчання | денна                           | Оцінка         |                              |
| Спеціальність  | 121                             |                |                              |
| 2026           |                                 |                |                              |

## Варіант завдання: Симулятор системи залежностей (Dependency resolver).

### Хід роботи:

#### Error Handling.

Створимо базовий клас для всіх винятків, що міститиме три конструктори: без повідомлення, з повідомленням, з повідомленням та внутрішнім виключенням:

```
/// <summary>
/// Represents errors that occur during package dependency processing.
/// Serves as the base class for all dependency-related exceptions.
/// </summary>
19 references
public class DependencyException : Exception
{
    /// <summary>
    /// Initializes a new instance of the <see cref="DependencyException"/> class.
    /// </summary>
    3 references
    public DependencyException()
    {
    }

    /// <summary>
    /// Initializes a new instance of the <see cref="DependencyException"/> class with a specified error message.
    /// </summary>
    /// <param name="message">The message that describes the error.</param>
    3 references
    public DependencyException(string message)
        : base(message)
    {
    }

    /// <summary>
    /// Initializes a new instance of the <see cref="DependencyException"/> class
    /// with a specified error message and a reference to the inner exception that is the cause of this exception.
    /// </summary>
    /// <param name="message">The error message that explains the reason for the exception.</param>
    /// <param name="innerException">The exception that caused the current exception.</param>
    3 references
    public DependencyException(string message, Exception innerException)
        : base(message, innerException)
    {
    }
}
```

Який наслідуватимуть три похідні класи, кожен з яких відповідає за один з типів виключень.

Циклічні залежності:

```

/// <summary>
/// Represents an error that occurs when a dependency cycle is detected in the package dependency graph.
/// </summary>
8 references
public class CyclicDependencyException : DependencyException
{
    /// <summary>
    /// Initializes a new instance of the <see cref="CyclicDependencyException"/> class.
    /// </summary>
    0 references
    public CyclicDependencyException()
    {
    }

    /// <summary>
    /// Initializes a new instance of the <see cref="CyclicDependencyException"/> class with a specified error message.
    /// </summary>
    /// <param name="message">The message that describes the error.</param>
    1 reference
    public CyclicDependencyException(string message)
        : base(message)
    {
    }

    /// <summary>
    /// Initializes a new instance of the <see cref="CyclicDependencyException"/> class
    /// with a specified error message and a reference to the inner exception that is the cause of this exception.
    /// </summary>
    /// <param name="message">The error message that explains the reason for the exception.</param>
    /// <param name="innerException">The exception that caused the current exception.</param>
    0 references
    public CyclicDependencyException(string message, Exception innerException)
        : base(message, innerException)
    {
    }
}

```

Некоректний формат:

```

/// <summary>
/// Represents an error that occurs when a package line does not conform to the expected format.
/// </summary>
12 references
public class PackageFormatException : DependencyException
{
    /// <summary>
    /// Initializes a new instance of the <see cref="PackageFormatException"/> class.
    /// </summary>
    0 references
    public PackageFormatException()
    {
    }

    /// <summary>
    /// Initializes a new instance of the <see cref="PackageFormatException"/> class with a specified error message.
    /// </summary>
    /// <param name="message">The message that describes the error.</param>
    4 references
    public PackageFormatException(string message)
        : base(message)
    {
    }

    /// <summary>
    /// Initializes a new instance of the <see cref="PackageFormatException"/> class
    /// with a specified error message and a reference to the inner exception that is the cause of this exception.
    /// </summary>
    /// <param name="message">The error message that explains the reason for the exception.</param>
    /// <param name="innerException">The exception that caused the current exception.</param>
    0 references
    public PackageFormatException(string message, Exception innerException)
        : base(message, innerException)
    {
    }
}

```

Відсутність пакету, наявність якого вимагає інший пакет:

```

/// <summary>
/// Represents an error that occurs when a package declares a dependency on another package that is missing from the dependency graph.
/// </summary>
10 references
public class MissingDependencyException : DependencyException
{
    /// <summary>
    /// Initializes a new instance of the <see cref="MissingDependencyException"/> class.
    /// </summary>
    0 references
    public MissingDependencyException()
    {
    }

    /// <summary>
    /// Initializes a new instance of the <see cref="MissingDependencyException"/> class with a specified error message.
    /// </summary>
    /// <param name="message">The message that describes the error.</param>
    1 reference
    public MissingDependencyException(string message)
        : base(message)
    {
    }

    /// <summary>
    /// Initializes a new instance of the <see cref="MissingDependencyException"/> class
    /// with a specified error message and a reference to the inner exception that is the cause of this exception.
    /// </summary>
    /// <param name="message">The error message that explains the reason for the exception.</param>
    /// <param name="innerException">The exception that caused the current exception.</param>
    0 references
    public MissingDependencyException(string message, Exception innerException)
        : base(message, innerException)
    {
    }
}

```

Тоді можемо таким чином кидати винятки:

```

if (string.IsNullOrWhiteSpace(name))
{
    throw new PackageFormatException("Package name is missing in line " + line);
}

```

А відловлювати винятки, вказавши в catch тільки базовий клас, і при виведенні помилки вже надавати інформацію про конкретний похідний клас, що викликав виняток:

```

private static void Main(string[] args)
{
    try
    {
        string filePath = args[0];

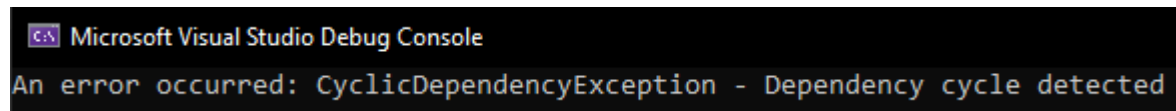
        var lines = new FileReader().ReadLines(filePath);
        DependencyGraph graph = new GraphBuilder().BuildGraph(lines);
        DependencyResolver resolver = new DependencyResolver(graph);
        List<Package> order = resolver.Resolve();

        Console.WriteLine("Package installation order:\n");

        for (int i = 0; i < order.Count; i++)
        {
            Console.WriteLine(i + 1 + ". " + order[i].name + "-" + order[i].version);
        }
    }
    catch (DependencyException ex)
    {
        Console.WriteLine("An error occurred: " + ex.GetType().Name + " - " + ex.Message);
    }
    catch (FileNotFoundException ex)
    {
        Console.WriteLine("File not found: " + ex.FileName);
    }
}

```

Тоді, наприклад, при спробі обробити файл з циклічною залежністю, виведеться повідомлення:



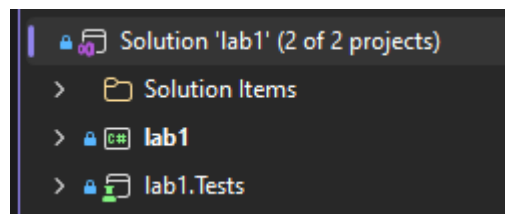
```

Microsoft Visual Studio Debug Console
An error occurred: CyclicDependencyException - Dependency cycle detected

```

## Unit Testing.

Скористаємось інструментом xUnit. Додамо тестовий проект до існуючого рішення:



Передбачимо тестування трьох класів: DependencyGraph (представляє граф та методи для роботи з ним), GraphBuilder (з рядків файлу створює граф), DependencyResolver (визначає порядок встановлення пакетів). Для кожного з цих класів існуватиме відповідний клас для тестування їхніх методів, що міститимуть позитивні та негативні тести:

```
> C# DependencyGraphTests.cs
> C# DependencyResolverTests.cs
> C# GraphBuilderTests.cs
```

DependencyGraph.

Метод для додавання пакету протестуємо коректним сценарієм. Перевіримо, що пакет присутній у списку:

```
[Fact]
0 references
public void AddPackage_AddsPackageToGraph()
{
    DependencyGraph graph = new DependencyGraph();
    Package packageA = new Package("A", "v1.0");

    graph.AddPackage(packageA);

    Assert.Contains(packageA, graph.GetAllPackages());
}
```

А також спробуємо додати два однакових пакети. При цьому програма не повинна цього допустити та просто проігнорувати це:

```
[Fact]
0 references
public void AddPackage_DoesNotDuplicatePackage()
{
    DependencyGraph graph = new DependencyGraph();
    Package packageA = new Package("A", "v1.0");
    Package packageADuplicate = new Package("A", "v1.0");

    graph.AddPackage(packageA);
    graph.AddPackage(packageADuplicate);

    Assert.Single(graph.GetAllPackages());
}
```

Для додавання залежностей також перевіримо коректний сценарій (спробуємо додати залежність між двома пакетами, перевіривши чи дійсно вона додалась):

```
[Fact]
0 references
public void AddDependency_AddsDependencyToGraph()
{
    DependencyGraph graph = new DependencyGraph();
    Package packageA = new Package("A", "v1.0");
    Package packageB = new Package("B", "v1.0");

    graph.AddPackage(packageA);
    graph.AddPackage(packageB);
    graph.AddDependency(packageA, packageB);

    Assert.Contains(packageB, graph.GetDependencies(packageA));
}
```

Перевіримо, чи не буде додано дві однакові залежності:

```
[Fact]
0 references
public void AddDependency_DoesNotDuplicateDependencies()
{
    DependencyGraph graph = new DependencyGraph();
    Package packageA = new Package("A", "v1.0");
    Package packageB = new Package("B", "v1.0");

    graph.AddPackage(packageA);
    graph.AddPackage(packageB);
    graph.AddDependency(packageA, packageB);
    graph.AddDependency(packageA, packageB);

    Assert.Single(graph.GetDependencies(packageA));
}
```

А також чи не може пакет залежати сам від себе. Така ситуація не повинна бути допущена:

```
[Fact]
0 references
public void AddDependency_DoesNotAllowSelfDependency()
{
    DependencyGraph graph = new DependencyGraph();
    Package packageA = new Package("A", "v1.0");

    graph.AddPackage(packageA);
    graph.AddDependency(packageA, packageA);

    Assert.Empty(graph.GetDependencies(packageA));
}
```

Отримання залежностей. Коректний сценарій. Додамо одну залежність пакету А, і перевіримо, чи дійсно кількість його залежностей буде рівною одиниці:

```
[Fact]
0 references
public void GetDependencies_ReturnsDependenciesForPackage()
{
    DependencyGraph graph = new DependencyGraph();
    Package packageA = new Package("A", "v1.0");
    Package packageB = new Package("B", "v1.0");

    graph.AddPackage(packageA);
    graph.AddPackage(packageB);
    graph.AddDependency(packageA, packageB);

    List<Package> dependencies = graph.GetDependencies(packageA);

    Assert.Single(dependencies, packageB);
}
```

Перевіримо, чи повертається порожній список залежностей, коли вказаний пакет їх не містить (1), або не існує (2):

```
[Fact]
0 references
public void GetDependencies_ReturnsEmptyListForPackageWithoutDependencies()
{
    DependencyGraph graph = new DependencyGraph();
    Package packageA = new Package("A", "v1.0");

    graph.AddPackage(packageA);
    List<Package> dependencies = graph.GetDependencies(packageA);

    Assert.Empty(dependencies);
}

[Fact]
0 references
public void GetDependencies_ReturnsEmptyListForNonExistentPackage()
{
    DependencyGraph graph = new DependencyGraph();
    Package packageA = new Package("A", "v1.0");

    List<Package> dependencies = graph.GetDependencies(packageA);

    Assert.Empty(dependencies);
}
```

Також перевіримо коректний сценарій роботи методу для отримання всіх пакетів. Перевіримо стандартний сценарій (1), та випадок з порожнім графом (2):



```

[Fact]
0 references
public void GetAllPackages_ReturnsAllAddedPackages()
{
    DependencyGraph graph = new DependencyGraph();
    Package packageA = new Package("A", "v1.0");
    Package packageB = new Package("B", "v1.0");

    graph.AddPackage(packageA);
    graph.AddPackage(packageB);
    graph.AddDependency(packageA, packageB);
    IEnumerable<Package> packages = graph.GetAllPackages();

    Assert.Equal(2, packages.Count());
    Assert.Contains(packageA, packages);
    Assert.Contains(packageB, packages);
}

[Fact]
0 references
public void GetAllPackages_ReturnsEmptyListForEmptyGraph()
{
    DependencyGraph graph = new DependencyGraph();
    IEnumerable<Package> packages = graph.GetAllPackages();

    Assert.Empty(packages);
}

```

GraphBuilder.

Спочатку негативні тести. Перевіримо, чи спрацює виключення при відсутності пакета, наявність якого вимагається іншим пакетом:

```

[Fact]
0 references
public void BuildGraph_ThrowsMissingDependencyException_WhenDependencyPackageMissing()
{
    List<string> lines = new List<string>
    {
        "package: A, v1.0",
        "requires: B, v1.0"
    };

    GraphBuilder builder = new GraphBuilder();

    Assert.Throws<MissingDependencyException>(() => builder.BuildGraph(lines));
}

```

Також перевіримо випадок, коли назва пакету співпадає, але версія відрізняється:

```

[Fact]
0 references
public void BuildGraph_ThrowsMissingDependencyException_WhenVersionMismatch()
{
    List<string> lines = new List<string>
    {
        "package: A, v1.0",
        "package: B, v1.0",
        "requires: A, v1.0",
        "package: C, v2.0",
        "requires: B, v2.0"
    };

    GraphBuilder builder = new GraphBuilder();

    Assert.Throws<MissingDependencyException>(() => builder.BuildGraph(lines));
}

```

Створимо параметризований тест для перевірки багатьох можливих варіацій некоректного формату рядків з пакетами:

```

[Theory]
[InlineData("package: ")]
[InlineData("package: A")]
[InlineData("package: A, ")]
[InlineData("package: , v")]
[InlineData("package: A, 1.0")]
[InlineData("package: A, v")]
0 references
public void BuildGraph_ThrowsPackageFormatException(string line)
{
    GraphBuilder builder = new GraphBuilder();
    List<string> lines = new List<string> { line };

    Assert.Throws<PackageFormatException>(() => builder.BuildGraph(lines));
}

```

Позитивні тести. Перевіримо метод для побудови графу на коректних даних. Всі три пакети повинні бути додані, а також їхні вказані залежності:

```

[Fact]
0 references
public void BuildGraph_CreatesGraphWithPackagesAndDependencies()
{
    List<string> lines = new List<string>
    {
        "package: A, v1.0",
        "package: B, v1.0",
        "requires: A, v1.0",
        "package: C, v2.0",
        "requires: B, v1.0"
    };

    GraphBuilder builder = new GraphBuilder();

    DependencyGraph graph = builder.BuildGraph(lines);
    IEnumerable<Package> packages = graph.GetAllPackages();

    Assert.Equal(3, packages.Count());
    Assert.Contains(packages, p => p.name == "A" && p.version == "v1.0");
    Assert.Contains(packages, p => p.name == "B" && p.version == "v1.0");
    Assert.Contains(packages, p => p.name == "C" && p.version == "v2.0");

    List<Package> dependenciesB = graph.GetDependencies(new Package("B", "v1.0"));
    Assert.Single(dependenciesB, new Package("A", "v1.0"));

    List<Package> dependenciesC = graph.GetDependencies(new Package("C", "v2.0"));
    Assert.Single(dependenciesC, new Package("B", "v1.0"));
}

```

Перевіримо, чи коректно обробляється пакет, що не містить залежностей:

```

[Fact]
0 references
public void BuildGraph_AddsPackageWithoutDependencies()
{
    List<string> lines = new List<string>
    {
        "package: A, v1.0"
    };

    GraphBuilder builder = new GraphBuilder();
    DependencyGraph graph = builder.BuildGraph(lines);

    Assert.Single(graph.GetAllPackages());
    Assert.Empty(graph.GetDependencies(new Package("A", "v1.0")));
}

```

Чи пропускаються рядки, що не відповідають очікуваному формату:

```

[Fact]
0 references
public void BuildGraph_IgnoresIncorrectLines()
{
    List<string> lines = new List<string>
    {
        "package: A, v1.0",
        "incorrect line",
        "package: B, v1.0",
        "requires: A, v1.0",
    };

    GraphBuilder builder = new GraphBuilder();
    DependencyGraph graph = builder.BuildGraph(lines);
    IEnumerable<Package> packages = graph.GetAllPackages();

    Assert.Equal(2, packages.Count());

    List<Package> dependencies = graph.GetDependencies(new Package("B", "v1.0"));
    Assert.Single(dependencies);
    Assert.Contains(dependencies, p => p.name == "A" && p.version == "v1.0");
}

```

Чи буде пропущено рядок з “requires:”, якщо перед ним не було “package:”:

```

[Fact]
0 references
public void BuildGraph_IgnoresRequiresWithoutPackage()
{
    var lines = new List<string>
    {
        "requires: A, v1.0",
        "package: A, v1.0"
    };

    GraphBuilder builder = new GraphBuilder();
    DependencyGraph graph = builder.BuildGraph(lines);

    Assert.Single(graph.GetAllPackages());
}

```

Чи пропуститься рядок з порожнім “requires:”:

```

[Fact]
0 references
public void BuildGraph_IgnoresEmptyRequires()
{
    List<string> lines = new List<string>
    {
        "package: A, v1.0",
        "requires: ",
        "package: B, v1.0"
    };

    GraphBuilder builder = new GraphBuilder();
    DependencyGraph graph = builder.BuildGraph(lines);
    IEnumerable<Package> packages = graph.GetAllPackages();

    Assert.Equal(2, packages.Count());
}

```

І чи буде порожній список, якщо граф не отримувал ніякого вмісту файлу:

```

[Fact]
0 references
public void BuildGraph_EmptyInput_ReturnsEmptyGraph()
{
    List<string> lines = new List<string>();

    GraphBuilder builder = new GraphBuilder();
    DependencyGraph graph = builder.BuildGraph(lines);

    Assert.Empty(graph.GetAllPackages());
}

```

DependencyResolver.

Перевіримо, чи буде виявлено циклічну залежність:

```
[Fact]
0 references
public void Resolve_ThrowsCyclicDependencyException()
{
    Package packageA = new Package("A", "v1.0");
    Package packageB = new Package("B", "v1.0");

    DependencyGraph graph = new DependencyGraph();
    graph.AddPackage(packageA);
    graph.AddPackage(packageB);
    graph.AddDependency(packageA, packageB);
    graph.AddDependency(packageB, packageA);

    DependencyResolver resolver = new DependencyResolver(graph);

    Assert.Throws<CyclicDependencyException>(() => resolver.Resolve());
}
```

Перевіримо, чи коректний порядок встановлення при звичайних вхідних даних:

```
[Fact]
0 references
public void Resolve_ReturnsCorrectOrder()
{
    Package packageA = new Package("A", "v1.0");
    Package packageB = new Package("B", "v1.0");
    Package packageC = new Package("C", "v1.0");

    DependencyGraph graph = new DependencyGraph();
    graph.AddPackage(packageA);
    graph.AddPackage(packageB);
    graph.AddPackage(packageC);
    graph.AddDependency(packageB, packageA);
    graph.AddDependency(packageC, packageB);

    DependencyResolver resolver = new DependencyResolver(graph);

    List<Package> order = resolver.Resolve();

    Assert.True(order.IndexOf(packageA) < order.IndexOf(packageB));
    Assert.True(order.IndexOf(packageB) < order.IndexOf(packageC));
}
```

Перевіримо порядок встановлення при наявності у пакета кількох залежностей:



```
[Fact]
✓ | 0 references
public void Resolve_ReturnsCorrectOrderWhenMultipleDependencies()
{
    Package packageA = new Package("A", "v1.0");
    Package packageB = new Package("B", "v1.0");
    Package packageC = new Package("C", "v1.0");

    DependencyGraph graph = new DependencyGraph();
    graph.AddPackage(packageA);
    graph.AddPackage(packageB);
    graph.AddPackage(packageC);
    graph.AddDependency(packageC, packageA);
    graph.AddDependency(packageC, packageB);

    DependencyResolver resolver = new DependencyResolver(graph);

    List<Package> order = resolver.Resolve();

    Assert.True(order.IndexOf(packageA) < order.IndexOf(packageC));
    Assert.True(order.IndexOf(packageB) < order.IndexOf(packageC));
}
```

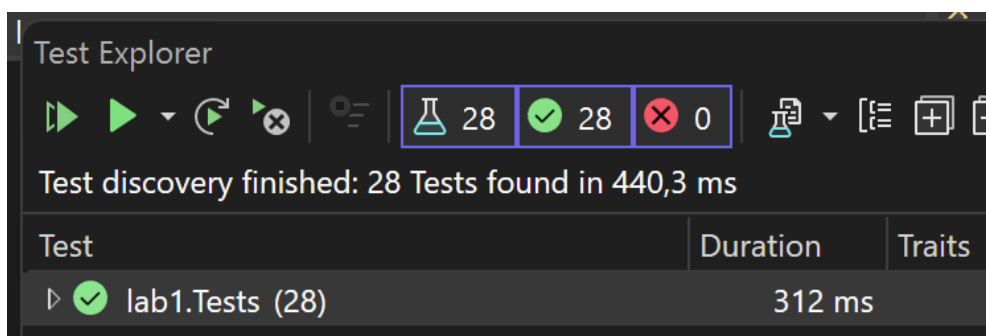
І коли граф порожній, то список повинен повернутись також порожнім:

```
[Fact]
✓ | 0 references
public void Resolve_ReturnsEmptyListIfGraphIsEmpty()
{
    DependencyGraph graph = new DependencyGraph();
    DependencyResolver resolver = new DependencyResolver(graph);

    List<Package> order = resolver.Resolve();

    Assert.Empty(order);
}
```

Запустимо тести, впевнившись, що всі вони пройшли:

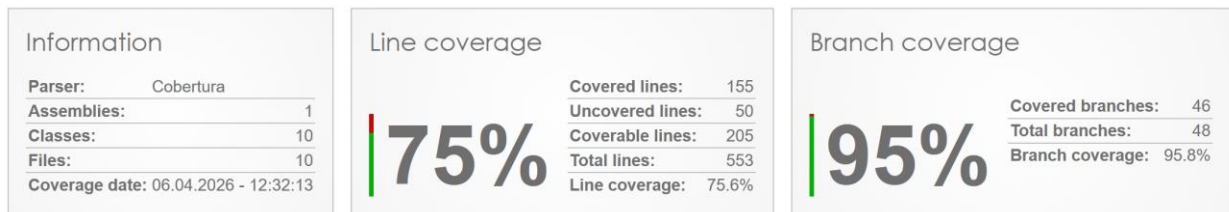


Test Explorer

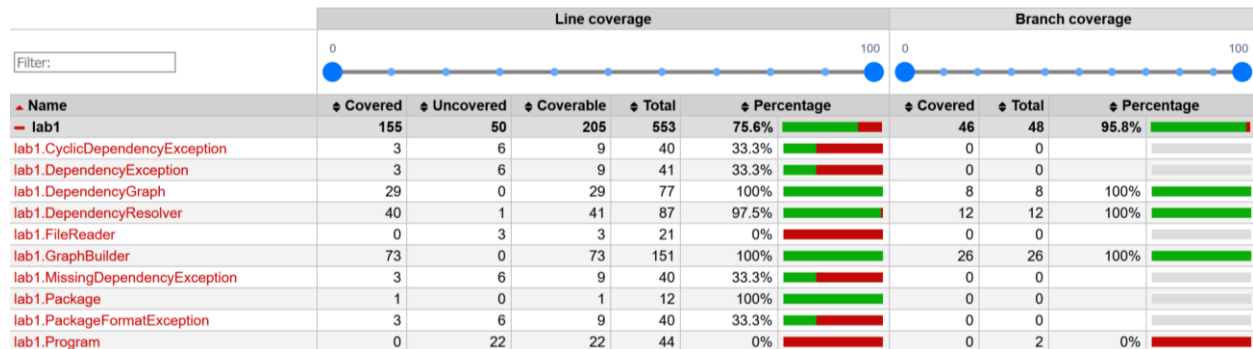
Test discovery finished: 28 Tests found in 440,3 ms

| Test            | Duration | Traits |
|-----------------|----------|--------|
| lab1.Tests (28) | 312 ms   |        |

Також можемо згенерувати html-звіт:



І впевнитись, що класи, відповідальні за ключову логіку, покриті фактично повністю:



## Висновки.

У межах цієї роботи було розглянуто два аспекти, відповідальні за коректну роботу програми: обробка виняткових ситуацій, розробка модульних тестів.

Було створено ієрархію класів, відповідальних за обробку виняткових ситуацій, де кожен клас відповідає за конкретні типи виключень. Створення ієрархії дозволило в блоці catch використовувати один базовий клас, замість того, щоб створювати декілька окремих catch блоків.

Створено модульні тести з використанням інструменту xUnit для класів, що містять ключову логіку програми. При розробці тестів також було виявлено певні недоліки в існуючому алгоритмі обробки вхідних даних, і усунено їх.